

Cost Optimization

Savings Plans · Right-Sizing · Reserved Instances · S3 Tiers · Budgets · FinOps

AWS Tutorial · ShopFlow

19-00

Chapter Overview — Cost Optimization at ShopFlow

AWS cost optimization is a continuous engineering discipline, not a one-time task. ShopFlow spends ~\$47,000/month on AWS at 1M MAU. This chapter covers seven levers: right-sizing compute, Savings Plans and Reserved Instances, Spot Instances, S3 storage tiers, data transfer optimization, database cost optimization, and automated cost governance. Goal: reduce to ~\$32,000/month (32% savings) without degrading reliability or performance.

Cost Category	Current Monthly Spend	% of Total	Optimization Target	Expected Savings
Compute (ECS Fargate, Lambda)	\$18,400	39%	Savings Plans + right-sizing + Spot	\$7,200 (39%)
Databases (Aurora, DynamoDB, ElastiCache)	\$12,600	27%	Reserved capacity + right-sizing	\$4,400 (35%)
Networking (data transfer, NAT Gateways, ALB)	\$6,200	13%	VPC endpoints + CloudFront + ALB cost optimization	\$2,500 (40%)
Storage (S3, EBS snapshots, ECR)	\$3,800	8%	Intelligent Tiering + lifecycle policies	\$1,500 (39%)
Observability (CloudWatch, X-Ray)	\$2,900	6%	Log retention + metric filters	\$800 (28%)
Other (Kinesis, SQS, Cognito, WAF)	\$3,100	7%	On-demand tuning + WAF rule optimization	\$600 (19%)

ShopFlow Cost Optimization Strategy

Step 1: Measure — Cost Explorer + Cost Allocation Tags (this sprint)

Step 2: Right-Size — CloudWatch metrics guide EC2/Fargate task sizing

Step 3: Commit — Savings Plans after 30 days of right-sized baselines

Step 4: Automate — Lambda schedules off-hours scaling; Budgets alerts

Step 5: Review — Monthly FinOps review; 5% quarter-over-quarter savings target

19-01

Cost Allocation Tags — The Foundation

Without cost allocation tags, AWS Cost Explorer shows total spend but cannot attribute costs to teams, services, or features. ShopFlow mandates four tags on every resource: Environment (prod/staging/dev), Service (order-svc/product-svc/etc.), Team (platform/backend/frontend), CostCenter (engineering/marketing/operations). CDK Aspects enforce tagging at synth time.

CDK Aspect — Enforce Cost Tags

```
class CostTaggingAspect implements IAspect {
  constructor(private readonly team: string,
    private readonly service: string) {}
  visit(node: IConstruct): void {
    if (node instanceof CfnResource) {
      Tags.of(node).add("Team", this.team);
      Tags.of(node).add("Service", this.service);
      Tags.of(node).add("Environment", process.env.ENVIRONMENT ?? "dev");
      Tags.of(node).add("CostCenter", "engineering");
    }
  }
}
```

```
Tags.of(node).add("ManagedBy", "CDK");
}
}
}
// Apply to all stacks in app
const app = new App();
const orderStack = new OrderServiceStack(app,"OrderStack",{env});
Aspects.of(orderStack).add(new CostTaggingAspect("backend","order-svc"));
```

Tag Key	Values	Purpose	Cost Explorer Filter
Environment	prod, staging, dev	Separate prod vs non-prod spend; dev should be <10% of prod	Env=prod: see production-only costs
Service	order-svc, product-svc, user-svc, search-svc	Attribute costs to each microservice	Service=order-svc: full cost of order process
Team	platform, backend, frontend, data	Charge back to engineering teams	Team=data: full data engineering cost (Kin
CostCenter	engineering, marketing, ops	Finance chargeback to business units	CostCenter=marketing: A/B test infrastru

19-02

AWS Compute Savings Plans — 39% Compute Discount

Savings Plans commit to a \$/hour spend on compute for 1 or 3 years in exchange for up to 66% discount vs on-demand. Compute Savings Plans apply to Fargate, Lambda, and EC2 regardless of family, size, region, or OS. ShopFlow baseline: \$18,400/month compute. After right-sizing: \$14,000/month. Commit \$8,000/month (57% of baseline) for 1 year = ~\$5,400 savings/month.

Savings Plan Type	Flexibility	Max Discount	ShopFlow Recommendation
Compute Savings Plan	Fargate + Lambda + EC2; any region, size, OS	Up to 66% vs on-demand (vs EC2 on-demand)	Yes. ShopFlow uses Fargate + Lambda
EC2 Instance Savings Plan	Specific EC2 family and region only	Up to 72% vs on-demand	Not recommended: ShopFlow primarily uses Fargate
SageMaker Savings Plan	SageMaker instances only	Up to 64%	Not applicable (ShopFlow uses Kine

Savings Plan Commitment Calculation

```
Current Fargate + Lambda monthly spend: $18,400
After right-sizing (remove over-provisioned tasks): $14,000
Savings Plan commitment strategy:
Commit: $8,000/month (57% of right-sized baseline)
Reason: leave 43% on-demand for Black Friday spikes
Savings Plans cover steady-state; on-demand covers spikes
Savings Plan discount (Compute SP, 1-year, no upfront): ~36%
On $8,000 committed: save $2,880/month = $34,560/year
Effective hourly rate: Fargate 1 vCPU: $0.05056 → $0.03236/hr (36% off)
Black Friday (10x, 3 days): on-demand Fargate at full price
Cost: 3 days x ~$5,000/day extra = $15,000 incremental (budgeted)
■ Always right-size before committing to Savings Plans. A Savings Plan commitment on over-provisioned tasks wastes money twice: you pay for unused capacity AND you lock in a commitment based on inflated baseline. Right-size for 30 days (CloudWatch CPU/Memory utilization metrics), then commit at 70-80% of the right-sized baseline. The remaining 20-30% on-demand handles day-to-day variance.
```

19-03

ECS Fargate Right-Sizing with CloudWatch

Fargate task sizing is the highest-impact right-sizing lever at ShopFlow. Common mistake: teams request 4 vCPU / 8GB for services that average 0.4 vCPU / 1.2GB utilization. Result: paying 10x more than necessary. ShopFlow audit: order-svc was over-provisioned by 60%. Right-sizing saves \$4,200/month on Fargate alone.

Service	Current Task Size	Avg CPU Usage	Avg Memory Usage	Right-Sized To	Monthly Savings
order-svc	2 vCPU / 4 GB	0.6 vCPU (30%)	1.8 GB (45%)	1 vCPU / 3 GB	\$1,440 (10 tasks)

Service	Current Task Size	Avg CPU Usage	Avg Memory Usage	Right-Sized To	Monthly Savings
product-svc	2 vCPU / 4 GB	0.3 vCPU (15%)	0.9 GB (22%)	0.5 vCPU / 2 GB	\$1,800 (12 tasks)
search-svc	4 vCPU / 8 GB	1.2 vCPU (30%)	3.2 GB (40%)	2 vCPU / 4 GB	\$1,080 (6 tasks)
user-svc	1 vCPU / 2 GB	0.2 vCPU (20%)	0.5 GB (25%)	0.25 vCPU / 1 GB	\$480 (8 tasks)

CloudWatch Metrics for Right-Sizing Decision

```
// Query: 30-day P95 CPU utilization for order-svc
aws cloudwatch get-metric-statistics
--namespace ECS/ContainerInsights
--metric-name CpuUtilized
--dimensions Name=ServiceName,Value=order-svc
Name=ClusterName,Value=shopflow-prod
--start-time 2024-10-01T00:00:00Z
--end-time 2024-10-31T00:00:00Z
--period 3600
--statistics p95
// Right-size rule: new_size = P95_usage x 1.5 (50% headroom)
// P95 CPU = 0.62 vCPU x 1.5 headroom = 0.93 → round up to 1 vCPU
// P95 Memory = 1.4 GB x 1.5 = 2.1 GB → round up to 3 GB (Fargate increments)
```

19-04 Aurora Reserved Instances — 35% Database Savings

Aurora Reserved Instances (RIs) provide up to 62% discount vs on-demand for 1-year commitment, 71% for 3-year. ShopFlow Aurora cluster: db.r6g.2xlarge writer + 2x db.r6g.xlarge readers. On-demand: \$4,320/month. 1-year no-upfront RI: \$2,808/month (35% savings). 3-year partial-upfront RI: \$1,858/month (57% savings).

RI Type	Upfront Payment	Monthly Cost	Total 1-Year	Savings vs On-Demand
On-Demand (no commitment)	\$0	\$4,320/mo	\$51,840	Baseline
1-Year No Upfront	\$0	\$2,808/mo	\$33,696	35% (\$18,144 saved)
1-Year Partial Upfront	\$12,000	\$1,944/mo	\$35,328	32% (\$16,512 saved)
1-Year All Upfront	\$31,200	\$0	\$31,200	40% (\$20,640 saved)
3-Year Partial Upfront	\$24,000 upfront	\$1,244/mo	~\$68,784 total (3yr)	57% annualized (\$29,376/yr saved)

Aurora RI Purchase CDK (via CloudFormation)

```
// Note: RIs are purchased in AWS Console or via CLI, not CDK
// CDK manages DB cluster size; RI discount applies automatically
// CLI: purchase Aurora RI
aws rds purchase-reserved-db-instances-offering
--reserved-db-instances-offering-id [find-via-describe-offerings]
--db-instance-count 1
--tags Key=Service,Value=shopflow Key=Purpose,Value=aurora-writer
// Find offering IDs for Aurora PostgreSQL db.r6g.2xlarge, 1-year, no-upfront:
aws rds describe-reserved-db-instances-offerings
--db-instance-class db.r6g.2xlarge
--product-description aurora-postgresql
--offering-type "No Upfront"
--duration 31536000
--query
"ReservedDBInstancesOfferings[].[ID:ReservedDBInstancesOfferingId, Cost:RecurringCharges]"
```

■ Purchase RIs for the minimum stable fleet. If ShopFlow normally runs 1 writer + 2 readers, purchase 3 RIs. Readers added for Black Friday (auto-scaling up to 5 readers) run on-demand — that is fine, as spikes are temporary. Buying RIs for the maximum (5 readers) wastes money for 362 days per year. Aurora Serverless v2 is an alternative: automatically scales from 0.5 ACU to 128 ACU; no RI needed; 15% more expensive than provisioned at sustained load but eliminates

right-sizing.

19-05

DynamoDB Cost Optimization — On-Demand vs Provisioned

ShopFlow DynamoDB tables: product catalog (read-heavy, stable), shopping cart (write-heavy, variable), order history (write-heavy, spiky). Provisioned with Auto Scaling is cheapest for predictable workloads; On-Demand eliminates capacity planning at 1.25x higher per-request cost. Cart table On-Demand makes sense (Black Friday 10x spike); product catalog Provisioned (predictable reads).

	Mode	Normal RCU/WCU	Black Friday	Monthly Cost	Recommendation
Product-catalog	Provisioned + Auto Scaling	500 RCU / 50 WCU	500 RCU / 100 WCU (stable)	\$118/mo	Keep Provisioned; stable
Shopping-cart	On-Demand	200-2000 WCU (variable)	2000-20000 WCU (10x spike)	\$280/mo normal; \$2,800/mo BB	On-Demand: avoid throttling
Order-history	Provisioned + Auto Scaling	100 RCU / 200 WCU	100 RCU / 2000 WCU peak	\$95/mo	Pre-scale WCU to 2000 b
User-sessions	TTL + On-Demand	Variable (login spikes)	Variable	\$45/mo	On-Demand + aggressive

DynamoDB Auto Scaling CDK

```
const productTable = new dynamodb.Table(this, "ProductCatalog", {
  billingMode: dynamodb.BillingMode.PROVISIONED, // cheaper for stable load
  readCapacity: 500,
  writeCapacity: 50,
});
productTable.autoScaleReadCapacity({minCapacity: 100, maxCapacity: 1000})
  .scaleOnUtilization({targetUtilizationPercent: 70});
productTable.autoScaleWriteCapacity({minCapacity: 50, maxCapacity: 500})
  .scaleOnUtilization({targetUtilizationPercent: 70});
// On-Demand for cart (variable Black Friday traffic):
const cartTable = new dynamodb.Table(this, "ShoppingCart", {
  billingMode: dynamodb.BillingMode.PAY_PER_REQUEST, // On-Demand
});
```

19-06

NAT Gateway Cost Reduction — VPC Endpoints

NAT Gateway is one of the most overlooked AWS cost drivers. ShopFlow originally routed all ECS-to-AWS-service traffic through NAT Gateway (\$0.045/GB processed). With 50 GB/hour of data flowing to S3, DynamoDB, SQS, and Kinesis, NAT Gateway cost: \$1,620/month. VPC Interface/Gateway Endpoints eliminate NAT Gateway data processing charges for those services.

Service	NAT Gateway Traffic/Month	NAT Cost	VPC Endpoint Type	Endpoint Cost	Savings
S3	500 GB/month (Firehose + ETL)	\$22.50	Gateway Endpoint	\$0 (free)	\$22.50/mo
DynamoDB	200 GB/month (app reads/writes)	\$9.00	Gateway Endpoint	\$0 (free)	\$9.00/mo
Kinesis	150 GB/month (stream events)	\$6.75	Interface Endpoint	\$14.40/mo (2 AZs)	Break-even at 320 GB/mo
SQS	100 GB/month (queue messages)	\$4.50	Interface Endpoint	\$14.40/mo	Break-even at 320 GB/mo
ECR (Docker pull)	50 GB/month (deployments)	\$2.25	Interface Endpoint	\$14.40/mo	Only worth it with >320 GB/mo

S3 and DynamoDB Gateway Endpoints CDK — Free

```
// Gateway endpoints: no hourly charge; eliminate NAT Gateway costs for S3/DynamoDB
vpc.addGatewayEndpoint("S3Endpoint", {
  service: ec2.GatewayVpcEndpointAwsService.S3,
  subnets: [{subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS}],
});
vpc.addGatewayEndpoint("DynamoDBEndpoint", {
  service: ec2.GatewayVpcEndpointAwsService.DYNAMODB,
```

```
subnets: [{subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS}],
});
// Result: S3 and DynamoDB traffic no longer goes through NAT Gateway
// NAT Gateway hourly charge ($0.045/hr x 3 AZs x 720hr = $97.20/mo) unchanged
// But NAT data processing charge eliminated for S3/DynamoDB traffic
```

19-07 S3 Storage Class Optimization — Intelligent Tiering

ShopFlow S3 buckets store: product images (frequently accessed), data lake Bronze (rarely accessed after 30 days), ECR layers (accessed on deployment only). S3 Intelligent Tiering automatically moves objects between Frequent Access, Infrequent Access, and Archive Instant Access tiers based on access patterns — no manual lifecycle rules needed for variable-access objects.

S3 Tier	Storage Cost	Access Pattern	Min Object Size	ShopFlow Use
Standard	\$0.023/GB	Frequent access (daily or more)	None	Product images served via CloudFront; high-cardinality metrics
Standard-IA	\$0.0125/GB	Infrequent (monthly or less); \$0.01/GB retrieval	128 KB	Order attachments (PDF receipts); access pattern unknown
Intelligent Tiering	\$0.023/GB Frequent + \$0.0125/GB IA + \$0.00099/GB Archive	Variable	128 KB (smaller = no data lake)	Product images, Bronze layer; access pattern unknown
Glacier Instant Retrieval	\$0.004/GB	Quarterly or less; millisecond retrieval	128 KB	Audit logs (CloudTrail, ALB access logs); kubernetes logs
Glacier Deep Archive	\$0.00099/GB	Annually or less; 12hr retrieval	N/A	Financial records 7-year retention; never accessed

S3 Lifecycle Rules CDK — Data Lake

```
dataLakeBucket.addLifecycleRule({
  id: "bronze-tiering",
  prefix: "raw/",
  transitions: [
    {storageClass: s3.StorageClass.INTELLIGENT_TIERING, transitionAfter: Duration.days(0)},
    {storageClass: s3.StorageClass.GLACIER, transitionAfter: Duration.days(90)},
    {storageClass: s3.StorageClass.DEEP_ARCHIVE, transitionAfter: Duration.days(365)},
  ],
  expiration: Duration.days(2555), // 7-year financial data retention
});
```

19-08 CloudWatch Log Cost Reduction

CloudWatch Logs is the third largest cost category at ShopFlow: \$2,400/month. Primary drivers: ECS container logs retained indefinitely (\$1,200/month storage), high-cardinality custom metrics (\$800/month), and VPC Flow Logs (\$400/month). Solution: enforce 30-day log retention, export historical logs to S3 (90% cheaper), switch high-cardinality metrics to EMF.

Log Source	Current Monthly Cost	Current Retention	Optimized Retention	Savings
ECS container logs (all services)	\$1,200/mo	Infinite (never expire)	30 days CloudWatch + export to S3	\$900/mo (75%)
Lambda logs	\$180/mo	Infinite	14 days	\$140/mo (78%)
ALB access logs	\$320/mo (CW metric filters)	7 days	7 days (keep for debugging)	\$0 (already optimal)
VPC Flow Logs	\$400/mo	Infinite	7 days CloudWatch + S3 export for Athena	\$300/mo (75%)
CloudWatch enhanced monitoring	\$280/mo	1 day (default)	1 day (keep for real-time debugging)	\$0

Log Retention CDK — All Log Groups

```
// CDK Aspect: enforce log retention on ALL log groups
class LogRetentionAspect implements IAspect {
  visit(node: IConstruct): void {
    if (node instanceof logs.CfnLogGroup) {
      if (!node.retentionInDays) {
```

```

node.retentionInDays = 30; // default: 30 days
}
}
}
}
Aspects.of(app).add(new LogRetentionAspect());

```

19-09

AWS Budgets and Cost Anomaly Detection

AWS Budgets sends alerts when actual or forecasted spend exceeds thresholds. Cost Anomaly Detection uses ML to detect unusual spend patterns. ShopFlow has five budgets: total monthly budget, per-environment budgets (prod vs dev), per-service budgets, Black Friday budget (separate, higher limit), and a RI/SP utilization budget (alert if utilization drops below 80%).

Budgets CDK

```

new budgets.CfnBudget(this, "MonthlyBudget", {
  budget: {
    budgetName: "shopflow-monthly-total",
    budgetType: "COST",
    timeUnit: "MONTHLY",
    budgetLimit: {amount: 50000, unit: "USD"},
    costFilters: {TagKeyValue: ["user:Environment$prod"]},
  },
  notificationsWithSubscribers: [{
    notification: {
      notificationType: "ACTUAL",
      comparisonOperator: "GREATER_THAN",
      threshold: 80, // alert at 80% of budget ($40,000)
      thresholdType: "PERCENTAGE",
    },
    subscribers: [{subscriptionType: "EMAIL",
      address: "finops@shopflow.com"}],
  }, {
    notification: {
      notificationType: "FORECASTED",
      comparisonOperator: "GREATER_THAN",
      threshold: 100, // alert if forecast exceeds budget
      thresholdType: "PERCENTAGE",
    },
    subscribers: [{subscriptionType: "SNS",
      address: alertTopic.topicArn}],
  }],
});

```

19-10

Spot Instances for Batch Workloads

Spot Instances provide up to 90% discount vs on-demand but can be reclaimed with 2-minute notice. ShopFlow uses Spot for: Glue ETL jobs (fault-tolerant, restartable), CodeBuild compilation (stateless), and development ECS tasks (non-critical). Never use Spot for: Aurora read replicas, order-processing Lambda triggers, or production ECS tasks handling live traffic.

Workload	Spot Suitable?	Reason	ShopFlow Implementation	Savings
Glue ETL nightly jobs	Yes	Spark checkpointing; job restarts from checkpoint	ECS batch job with Spot + on-demand mix (80/20)	~70% on Glue DPU cost
CodeBuild compilation	Yes	Stateless; retry on failure; no user impact	CodeBuild Spot fleet	~70% on build cost
Dev/staging ECS tasks	Yes	Non-production; team tolerates interruptions	ECS capacity provider: Spot preferred, on-demand fallback	~65% on dev/staging compute
Production order-svc ECS	No	Customer-facing; 2-min Spot reclaim = dropped orders	Aurora on-demand (covered by Savings Plans)	~10% use Savings Plans

Workload	Spot Suitable?	Reason	ShopFlow Implementation	Savings
Aurora read replicas	No	DB state not safe to lose; reader failover disrupts queries	Reserved on-demand instances	N/A — use Reserved

ECS Spot Capacity Provider CDK (Dev Environment)

```
const spotCapacityProvider = new ecs.AsgCapacityProvider(this,"SpotProvider",{
  autoScalingGroup: new autoscaling.AutoScalingGroup(this,"SpotAsg",{
    vpc,
    instanceType: ec2.InstanceType.of(ec2.InstanceClass.M5, ec2.InstanceSize.XLARGE),
    spotPrice: "0.08", // max bid: $0.08/hr (on-demand is $0.192)
    minCapacity: 0, maxCapacity: 20,
  }),
  spotInstanceDraining: true, // 2-min draining when Spot reclaimed
});
```

19-11 Data Transfer Cost Optimization

AWS data transfer costs are hidden and complex. ShopFlow data transfer analysis: (1) Cross-AZ transfer (AZ-to-AZ) costs \$0.01/GB each direction — ECS service calling Aurora in different AZ = \$0.02/GB round trip. (2) Internet egress: \$0.09/GB for first 10TB, reduced via CloudFront (\$0.0085/GB). (3) S3 to EC2 in same region: free via Gateway Endpoint.

Transfer Type	Cost	Volume/Month	Monthly Cost	Optimization
Cross-AZ (ECS to Aurora different AZ)	\$0.02/GB each direction	500 GB	\$10.00	Aurora Multi-AZ: always same-AZ reads via reader endpoint
Internet egress (API responses to users)	\$0.09/GB (first 10TB), \$0.0085/GB thereafter	2,000 GB	\$180.00	CloudFront: \$0.0085/GB CDN = \$17/mo; 90% savings
S3 to ECS (same region, no VPC endpoint)	\$0.00/mo	500 GB	\$0	S3 Gateway Endpoint: always free; ensure all ECS tasks use same AZ
RDS cross-AZ replication	No charge	Continuous	\$0	Aurora replication is free (internal AWS network)

```
// Ensure all ECS tasks use same AZ as primary Aurora writer
// Aurora writer endpoint resolves to writer AZ (us-east-1a typically)
// Configure ECS tasks affinity to prefer us-east-1a:
taskDefinition.addPlacementConstraint(
  ecs.PlacementConstraint.memberOf("attribute:ecs.availability-zone == us-east-1a")
);
// Warning: reduces availability (all tasks in one AZ)
// ShopFlow decision: use reader endpoint for reads (auto-routes to healthy reader in any AZ)
// Only writer connections prefer same AZ; acceptable tradeoff
```

19-12 ElastiCache Reserved Nodes — Redis Cost Savings

ShopFlow ElastiCache Redis cluster: 3x cache.r6g.large nodes (cluster mode, 3 shards). On-demand cost: \$648/month. 1-year no-upfront reserved nodes: \$421/month (35% savings). Redis Reserved Nodes work like RDS RIs: commit to specific node type and term for a discount. Reservation is per-node (purchase 3 reserved nodes for 3-node cluster).

ElastiCache Node	On-Demand/mo	1-Year No-Upfront/mo	Savings	ShopFlow Commitment
cache.r6g.large (x3 nodes)	\$216/node = \$648/mo	\$140/node = \$421/mo	35% (\$227/mo = \$2,724/yr)	3 reserved nodes; match cluster size
cache.r6g.xlarge (x3 nodes)	\$432/node = \$1,296/mo	\$281/node = \$843/mo	35% (\$453/mo = \$5,436/yr)	3 reserved nodes; right-sized to xlarge after load testing

ElastiCache Right-Sizing — CloudWatch Metrics

```
// Check if cache.r6g.large is right-sized for ShopFlow:
// Key metrics to monitor before committing to reserved nodes:
// 1. SwapUsage: should be 0 (high swap = need more memory)
// 2. DatabaseMemoryUsagePercentage: target < 75%
// 3. CurrConnections: total active connections
```



```
// 4. CacheHitRate: target > 95% (low = cache too small or wrong keys)
// 5. CPUUtilization: target < 80% (Redis is single-threaded per shard)
aws cloudwatch get-metric-statistics
--namespace AWS/ElastiCache
--metric-name DatabaseMemoryUsagePercentage
--dimensions Name=CacheClusterId,Value=shopflow-redis-0001-001
--start-time 2024-10-01T00:00:00Z
--end-time 2024-10-31T00:00:00Z
--period 3600 --statistics Maximum
# If Max < 65%: downsize to cache.r6g.medium (saves additional 45%)
# If Max > 80%: upsize to cache.r6g.xlarge before reserving
```

19-13

Lambda Cost Optimization — Memory and Duration

Lambda pricing: \$0.0000166667 per GB-second. A 1 GB Lambda running 1 second costs the same as a 2 GB Lambda running 0.5 seconds. However, more memory gives proportionally more CPU — a 2 GB Lambda often completes in 40% of the time of a 1 GB Lambda, making it the same cost or cheaper. Use AWS Lambda Power Tuning to find the optimal memory setting.

ShopFlow Lambda	Current Memory	Duration P50	GB-s/invocation	Optimal Memory	Optimized GB-s	Savings
OrderProcessor (Kinesis consumer)	1024 MB	800ms	0.82	1536 MB (more CPU)	0.55 (faster)	33%
ProductSearch indexer	2048 MB	3200ms	6.55	1024 MB (not CPU-bound)	3.28 (same speed)	50%
Email notification sender	512 MB	2100ms	1.08	256 MB (I/O wait dominated)	1.08 (same speed)	100% (wait time)
DynamoDB stream processor	512 MB	150ms	0.08	256 MB (minimal memory usage)	0.04	50%

Lambda Power Tuning CDK (SAR deployment)

```
// AWS Lambda Power Tuning: SAR app that tests Lambda at multiple memory levels
// Deploys a Step Functions state machine that invokes Lambda at 128/256/512/1024/2048 MB
// Reports: cost vs performance curve; find optimal memory for each function
// Run after deployment:
// 1. Deploy SAR app: aws serverlessrepo create-cloud-formation-change-set ...
// 2. Invoke state machine with Lambda ARN and test payload
// 3. Read output: {power: 1536, cost: 0.0000092, duration: 550, stateMachine: "..."}
// 4. Update Lambda memory to optimal value
// 5. Measure 30-day actual invocations at new memory setting
```

■ Lambda ARM64 (Graviton2) architecture provides up to 34% better price-performance than x86. Cost is 20% cheaper per GB-second on ARM64. ShopFlow Lambda functions built with Java 21 on Lambda work identically on ARM64 — just change the architecture in CDK: `lambda.Architecture.ARM_64`. Apply to all Lambda functions immediately for instant 20% Lambda cost reduction with no code changes required.

19-14

Cost Explorer — Analyzing ShopFlow Spend

AWS Cost Explorer provides 13 months of cost data with daily granularity. Key Cost Explorer reports for ShopFlow: (1) Service breakdown by month, (2) Per-tag spend (identify top cost services), (3) RI and Savings Plan utilization and coverage, (4) Cost by linked account (staging vs prod). Use Cost Explorer API for programmatic reporting and Slack integration.

Cost Explorer API — Monthly Report

```
// Python: generate monthly cost report by service and tag
import boto3, json
ce = boto3.client("ce", region_name="us-east-1")
response = ce.get_cost_and_usage(
    TimePeriod={"Start": "2024-11-01", "End": "2024-11-30"},
    Granularity="MONTHLY",
    Filter={
```



```

"Tags": {
  "Key": "Environment",
  "Values": ["prod"],
},
},
GroupBy=[
  {"Type": "DIMENSION", "Key": "SERVICE"},
  {"Type": "TAG", "Key": "Service"},
],
Metrics=["BlendedCost", "UsageQuantity"],
)
# Output: breakdown by AWS Service + ShopFlow Service tag
for result in response["ResultsByTime"][0]["Groups"]:
  print(result["Keys"], result["Metrics"]["BlendedCost"]["Amount"])

```

■ Cost Explorer "Rightsizing Recommendations" provides EC2-specific suggestions but does not cover Fargate tasks. For Fargate right-sizing, use Container Insights metrics (CpuUtilized, MemoryUtilized) queried via CloudWatch Metrics Insights. Run a Logs Insights query across all ECS log groups: calculate per-task P95 CPU and memory over 30 days. Build a CDK construct that reads these metrics and automatically adjusts task definition CPU/memory if P95 utilization falls below 40% or exceeds 80%.

19-15

CloudFront Cost Optimization — Egress Reduction

CloudFront reduces ShopFlow internet egress costs from \$0.09/GB to \$0.0085/GB for cached content (91% reduction). ShopFlow serves 2 TB/month of API responses and product images. Without CloudFront: \$180/month egress. With CloudFront (\$0.0085/GB): \$17/month + CloudFront data transfer fee (\$0.0085/GB for first 10TB = \$17/month). Total: \$34/month vs \$180/month.

Content Type	Cache-Control TTL	CloudFront Cache Hit Rate	Monthly Egress Saved	Notes
Product images (S3 origin)	max-age=86400 (1 day)	95%+	~\$110/mo	S3 to CloudFront: free; CloudFront to internet: \$0.0085/GB
Product catalog API responses	max-age=60 (1 min) + stale-while-revalidate=300	70%	~\$40/mo	Short TTL: catalog changes every few days
Order history API	Cache-Control: no-store (do not cache)	0%	\$0 (intentionally uncached)	Order data is user-specific and real-time
JS/CSS/HTML assets (S3 origin)	max-age=31536000 (1 year, immutable) + hash in filenames	99%+	~\$15/mo	Hash in filename (app.a3f9b2.js); 1-year TTL

CloudFront Cache Policy CDK

```

const productApiCachePolicy = new cloudfront.CachePolicy(this, "ProductApiCache", {
  defaultTtl: Duration.seconds(60),
  minTtl: Duration.seconds(0),
  maxTtl: Duration.seconds(300),
  enableAcceptEncodingGzip: true,
  enableAcceptEncodingBrotli: true,
  headerBehavior: cloudfront.CacheHeaderBehavior.none(),
  queryStringBehavior: cloudfront.CacheQueryStringBehavior.allowList(
    ["category", "page", "limit"], // include query params in cache key
  );
});

```

19-16

FinOps Practice — Monthly Cost Review Process

FinOps (Financial Operations) applies DevOps principles to cloud cost management. ShopFlow FinOps monthly meeting: 30 minutes, engineering leads + finance. Agenda: (1) last month actual vs budget, (2) top 3 cost anomalies, (3) RI/SP utilization review, (4) one optimization task per team. Goal: 5% quarter-over-quarter cost reduction per service while maintaining SLA.

FinOps Metric	Target	Current ShopFlow	Action if Off Target
RI/SP Utilization	> 80%	72% (some Fargate tasks removed but SP still over-provisioned)	SP still over-provisioned than current: consolidate tasks;

FinOps Metric	Target	Current ShopFlow	Action if Off Target
RI/SP Coverage (% of compute covered by commitments)	> 60%	55%	Purchase additional Compute SP after next right
Untagged resource spend	< 2% of total	8% (legacy Lambda functions without tags)	CSK Aspect + lambda audit sprint; tag or decom
Dev/staging spend as % of production	< 10%	22% (dev ECS tasks running 24/7)	Schedule Lambda stops non-prod ECS at 7pm, r
Cost per transaction (order processed)	< \$0.012	\$0.019	Right-size order-svc tasks; reduce Lambda mem

Automated Dev Environment Shutdown

```
// Lambda: stop dev ECS services at 7pm ET, restart at 8am ET
const stopDevFn = new lambda.Function(this,"StopDevServices",{
  runtime: lambda.Runtime.PYTHON_3_12,
  handler: "index.handler",
  code: lambda.Code.fromInline(`
import boto3
def handler(event, context):
  ecs = boto3.client("ecs")
  for svc in ["order-svc","product-svc","user-svc"]:
    ecs.update_service(cluster="shopflow-dev",service=svc,desiredCount=0)
  `),
});
// EventBridge rule: run at 7pm ET (23:00 UTC) Mon-Fri
new events.Rule(this,"StopDevRule",{
  schedule: events.Schedule.cron({hour:"23",minute:"0",weekDay:"MON-FRI"}),
  targets: [new targets.LambdaFunction(stopDevFn)],
});
// Saves: 13 hours/night x 5 days/week x 4 weeks = 260 hours/month
// At $0.05056/vCPU-hr x 3 tasks x 2 vCPU each = $78.87/mo savings
```

19-17 Trusted Advisor and Compute Optimizer

AWS Trusted Advisor (Business/Enterprise support tier) provides automated checks across cost, performance, security, fault tolerance, and service limits. AWS Compute Optimizer uses ML to analyze EC2, Lambda, ECS, and Auto Scaling utilization and recommend optimal resource sizing. ShopFlow uses Compute Optimizer for Lambda memory recommendations and Trusted Advisor for idle resource detection.

Tool	Coverage	Access Tier	ShopFlow Findings	Action Taken
Trusted Advisor — Low Utilization	EC2 instances with < 10% CPU	Business Support or higher	8 NAT Gateway instances in dev (always on)	Revised dev NAT GW; use ECS B
Trusted Advisor — Idle Load Balancers	ALBs with no healthy targets	Business Support or higher	1 staging ALB left running during holidays	Deleted stage ALB; recreate via C
Compute Optimizer — Lambda	Optimal memory for each Lambda	Free (as of Jan 2024)	Search Lambda: 2048 MB	Reduced 24 MB to 50 MB
Compute Optimizer — ECS Fargate	CPU and memory right-sizing	Free (task definition)	Compute Optimizer 2 vCPU → 1 vCPU; 4 GB	Updated task definition in CDK; val

Enable Compute Optimizer CDK

```
// Enable AWS Compute Optimizer for all supported resource types
new computeoptimizer.CfnRecommendationPreferences(this,"ComputeOptimizerPrefs",{
  resourceType: "EcsService",
  scope: {name: "AccountId", value: this.account},
  enhancedInfrastructureMetrics: "Active", // 93-day lookback (vs 14-day default)
});
// Note: Compute Optimizer recommendations available in console after 14 days
// CLI: aws compute-optimizer get-ecs-service-recommendations
// --service-arns arn:aws:ecs:us-east-1:123456789:service/shopflow/order-svc
```

19-18 ECR Lifecycle Policies — Container Image Cost

ECR stores Docker images at \$0.10/GB-month. ShopFlow runs 12 microservices, each pushing a new 800MB image per deployment (10 deployments/day). Without lifecycle policies: $10 \times 12 \times 0.8 \text{ GB} \times 30 \text{ days} \times \$0.10 = \$288/\text{month}$ growing unboundedly. With lifecycle policy (keep last 10 images per service): $\$0.10 \times 10 \times 12 \times 0.8 \text{ GB} = \$9.60/\text{month}$.

ECR Lifecycle Policy CDK

```
const orderSvcRepo = new ecr.Repository(this,"OrderSvcRepo",{
  repositoryName: "shopflow/order-svc",
  lifecycleRules: [
    {
      rulePriority: 1,
      description: "Keep last 10 tagged images (for rollback)",
      tagStatus: ecr.TagStatus.TAGGED,
      tagPrefixList: ["v"], // tagged images: v1.2.3, v1.2.4 etc
      maxImageCount: 10,
    },
    {
      rulePriority: 2,
      description: "Delete untagged images after 1 day (leftover from builds)",
      tagStatus: ecr.TagStatus.UNTAGGED,
      maxImageAge: Duration.days(1),
    },
  ],
  removalPolicy: cdk.RemovalPolicy.RETAIN, // do not delete repo on CDK destroy
  imageScanOnPush: true, // ECR Basic scanning: no extra cost
});
```

■ ECR Enhanced Scanning (Inspector-based) costs \$0.09 per image scan vs Basic Scanning (free). For ShopFlow 12 repos x 10 images/day = 120 scans/day = \$10.80/day = \$324/month. Basic scanning is sufficient for most teams; enable Enhanced Scanning only for images deployed to production (not dev/staging builds). Use CodePipeline manual approval gate to gate production deployments on Basic scan results.

19-19 Cost Optimization — Consolidated Billing and Linked Accounts

ShopFlow uses AWS Organizations with separate accounts for: Production, Staging, Development, Security/Audit, and Shared Services. Consolidated billing aggregates usage across all accounts for volume discounts (S3, data transfer, CloudFront). Staging and Dev accounts share a single Savings Plan purchased in the management account. Service Control Policies (SCPs) prevent Dev from launching expensive instance types.

Account	Monthly Spend	Key Resources	Cost Controls	Budget Alert Threshold
Production	\$43,500	ECS Fargate, Aurora, Kinesis, ElastiCache, OpenSearch	Savings Plans + RIs; Budget: \$50K/mo	Alert at 80% (\$40K)
Staging	\$3,200	ECS Fargate (smaller tasks), Aurora (single-AZ), no ElastiCache	ECR Basic Scanning; budget \$4K/mo	Alert at 90% (\$3.6K)
Development	\$1,800	ECS Fargate (dev tasks, stop at 7pm), Aurora Dev	SCP: 4 max t4g.large; auto-stop ECS at 7pm	Budget: \$2K (\$1.6K)
Shared Services	\$600	CodePipeline, ECR, Route 53, CloudTrail	Fixed costs; minimal variance	Alert at 100% (\$600)

SCP — Restrict Dev Account Instance Types

```
// Service Control Policy: prevent large instances in Dev account
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyLargeInstances",
    "Effect": "Deny",
    "Action": ["ec2:RunInstances","ecs:RegisterTaskDefinition"],
    "Resource": "*",
    "Condition": {
      "StringLike": {
        "ec2:InstanceType": ["*.4xlarge","*.8xlarge","*.16xlarge","*.metal"]
      }
    }
  }]
}
```

```
}
}
}]
}
```

19-20 Graviton3 Migration — 20% Better Price-Performance

AWS Graviton3 (ARM64) provides 20% better price-performance than x86 (Intel/AMD) for most workloads. Fargate Graviton3 is 20% cheaper per vCPU-hour at the same performance. ShopFlow migrating all ECS tasks and Lambda functions to ARM64: estimated \$3,600/month savings. Java 21 runs identically on ARM64 — rebuild Docker images with --platform linux/arm64 flag.

Fargate ARM64 Migration CDK

```
const taskDefinition = new ecs.FargateTaskDefinition(this, "OrderSvcTask", {
  cpu: 1024,
  memoryLimitMiB: 3072,
  runtimePlatform: {
    operatingSystemFamily: ecs.OperatingSystemFamily.LINUX,
    cpuArchitecture: ecs.CpuArchitecture.ARM64, // Graviton3
  },
});
// Lambda ARM64:
const orderProcessorFn = new lambda.Function(this, "OrderProcessor", {
  runtime: lambda.Runtime.JAVA_21,
  architecture: lambda.Architecture.ARM_64, // 20% cheaper per GB-s
  memorySize: 1536,
  handler: "com.shopflow.OrderProcessorHandler::handleRequest",
});
```

Migration Step	Action	Risk	Validation
Build ARM64 Docker image	Add --platform linux/arm64 to docker build; push to ECR	Build fails; no binaries	Existing unit tests pass on ARM64
Deploy to staging (ARM64 Fargate)	CDK task definition to ARM64; deploy to staging	Identical to production	Load test: 30 min at 1000 req/s; compare
Deploy to production (one canary deployment)	10% → 50% → 100% traffic to ARM64 tasks	Rollback	Monitoring CPU latency and error rate during
Lambda ARM64	Change lambda.Architecture to ARM_64; redeploy	Early Lambda functions time handles	ARM64 compatibility duration and error

19-21 Savings Plan Utilization and Coverage Monitoring

Purchasing a Savings Plan does not guarantee savings — if ShopFlow reduces Fargate tasks below the committed spend level, the SP commitment still charges the committed \$/hour. SP Utilization: percentage of commitment actually consumed (target >90%). SP Coverage: percentage of eligible compute spend covered by commitments (target >60%). Monitor both weekly; adjust task counts to maximize utilization.

	Formula	ShopFlow Target	Current	Action
on	Actual SP-eligible spend / Committed SP spend x 100	>90%	72% (over-committed after task reduction)	Add ECS tasks or reduce
SP coverage	SP-covered spend / Total eligible spend x 100	>60%	55% (under-committed)	Purchase additional Comp
On-Demand waste	On-Demand spend on stable baseline compute	<\$1,000/mo on steady	\$1,200/mo (8 ECS tasks running on-demand)	Covered by additional SP

SP Utilization Report — Cost Explorer API

```
response = ce.get_savings_plans_utilization(
  TimePeriod={"Start": "2024-11-01", "End": "2024-11-30"},
  Granularity="MONTHLY",
)
util = response["Total"]
print(f"SP Utilization: {util["Utilization"]["UtilizationPercentage"]}%)")
```

```
print(f"Net Savings: ${util["Savings"]["NetSavings"]}",
print(f"On-Demand Equiv: ${util["Utilization"]["EstimatedOnDemandCostWithCurrentCommitment"]}")
# Example output:
# SP Utilization: 72.3%
# Net Savings: $2,045.60
# On-Demand Equiv: $5,888.00
# (if 100% utilized, net savings would be $2,833/month)
```

■ SP commitment optimal strategy: commit at 70% of right-sized baseline (not 100%). This leaves 30% on-demand headroom for: day-to-day traffic variance, new service launches, and Black Friday spike (SP covers baseline; on-demand covers burst). Never commit more than you run on the worst traffic day of your slowest month — Black Friday is the high, not the baseline.

19-22

Cost Optimization Roadmap — ShopFlow 90-Day Plan

ShopFlow 90-day cost optimization roadmap targeting \$15,000/month savings (32% reduction from \$47,000/month baseline). Month 1: Quick wins (tags, log retention, S3 lifecycle, gateway endpoints). Month 2: Right-size then commit (Fargate, Aurora, ElastiCache RIs). Month 3: Architectural changes (Graviton3, CloudFront expansion, dev shutdown automation).

Phase	Timeline	Actions	Expected Savings/Month	Effort
Phase 1: Quick Wins	Month 1 (Days 1-30)	Cost tags (CDK Aspect), 30-day log retention, S3 Intelligent Tiering	\$2,400/month	1 week
Phase 2: Right-Sizing	Month 2 (Days 31-60)	30 days of CloudWatch P95 data collection; resize Fargate tasks (\$4,800/mo 2vCPU to 1vCPU)	\$4,800/month	2 weeks
Phase 3: Commitment	Month 2-3 (Days 45-75)	Compute Savings Plan at \$8K/month (after right-sizing); Aurora RI (\$5,200/mo), ElastiCache reserved instances (\$8,000/mo)	\$13,000/month	3 weeks
Phase 4: Architectural	Month 3 (Days 61-90)	Graviton3 for all ECS + Lambda; dev environment auto-shutdown; CloudFront cache expansion; high-NR S3 Glacier	\$2,600/month	3 weeks

Running Total	Month 1 End	Month 2 End	Month 3 End
Monthly Spend	\$44,600	\$39,800	\$34,600
Savings vs Baseline (\$47K)	\$2,400 (5%)	\$7,200 (15%)	\$12,400 (26%)
Investment (engineer-weeks)	1 week	3 weeks	6 weeks total
ROI (savings per engineer-week)	\$2,400/week	\$2,400/week	\$2,067/week

19-23

Cost Optimization Anti-Patterns — What Not To Do

Cost optimization failures at ShopFlow and other engineering teams share common patterns: optimizing the wrong things (spending engineer time on \$50/month items vs \$5,000/month items), under-investing in measurement (optimizing without Cost Explorer = guessing), and making trade-offs that reduce reliability to save small amounts. A 99.9% SLA violation costs more in customer churn than \$200/month saved on redundant infrastructure.

Anti-Pattern	Cost	Correct Approach	ShopFlow Example
Optimizing small items first	Engineer spends 2 weeks optimizing SQS polling interval	Cost Explorer \$50/mo, spend \$4,200/mo on EC2	Fixed EC2 over-provisioning (\$4,200/mo)
Committing before right-sizing	Purchases 1-year Aurora RI for db.r6g.4xlarge; right-sizing shows db.r6g.2xlarge before any	Right-size first, then commit	ShopFlow only 60-day 2-month test collection
Removing Multi-AZ for cost savings	Removes Aurora Multi-AZ reader (\$432/mo savings)	Multi-AZ for high-traffic services	Multi-AZ for all prod services
Ignoring data transfer costs	Focuses on compute and storage; misses \$1,620/mo data transfer	Amazon Athena gateway data processing	ShopFlow Explorer S3 to Redshift via CloudFront
No tagging = no attribution	30% of AWS spend untagged; cannot identify which CDK Aspect for cost tags at spike	CDK Aspect for cost tags	CDK Tagging Aspect

19-24

Cost Optimization for Black Friday — Budget and Controls

Black Friday requires a separate cost budget and controls. ShopFlow Black Friday cost profile: 3 days at 10x normal traffic. Total incremental cost: \$42,000 (3 days x \$14,000/day vs normal \$1,567/day). This is planned and budgeted.

Cost controls: separate AWS Budget with \$50,000 limit for the Black Friday window, automatic alerting at 60% spend, pre-approved on-demand capacity, and post-event cost review.

Cost Category	Normal Daily	Black Friday Daily (10x)	3-Day BF Total	Notes
Fargate (ECS tasks)	\$613/day	\$4,200/day (5x tasks at higher traffic)	\$12,600	Savings Plan covers baseline; on-demand capacity for spike
Aurora (reader replicas)	\$144/day	\$576/day (4 readers vs 2 normal)	\$1,728	Pre-provision 2 extra readers day before
Kinesis (50 shards vs 8)	\$72/day	\$540/day (pre-scaled to 50 shards)	\$1,620	On-Demand mode would cost \$2,700
ElastiCache (extra nodes)	\$72/day	\$216/day (3 extra nodes)	\$648	Pre-provision 3 extra cache nodes
CloudFront (10x traffic)	\$17/day	\$170/day (10x requests, cached)	\$510	Cache helps: most product pages static
WAF (10x requests)	\$5/day	\$50/day	\$150	Rate limiting protects origin; WAF-CAESAR

Black Friday Budget CDK

```
// Separate budget for Black Friday window (Nov 27-30, 2024)
new budgets.CfnBudget(this,"BlackFridayBudget",{
  budget: {
    budgetName: "shopflow-black-friday-2024",
    budgetType: "COST",
    timeUnit: "MONTHLY",
    timePeriod: {start: "2024-11-01", end: "2024-11-30"},
    budgetLimit: {amount: 100000, unit: "USD"}, // Nov: normal + BF spike
    costFilters: {TagKeyValue: ["user:Environment$prod"]},
  },
  notificationsWithSubscribers: [{
    notification: {
      notificationType: "ACTUAL",
      comparisonOperator: "GREATER_THAN",
      threshold: 60, // alert at $60K (early BF warning)
      thresholdType: "PERCENTAGE",
    },
    subscribers: [{subscriptionType: "EMAIL",
      address: "cto@shopflow.com"}],
  }],
});
```

19-BP

Best Practices and Anti-Patterns

- ✓ Right-size before committing — 30 days of CloudWatch P95 utilization data before any RI or Savings Plan purchase; committing to over-provisioned sizes wastes money
- ✓ Enforce cost allocation tags via CDK Aspects at synth time — prevents untagged resources from reaching production; untagged spend cannot be attributed to teams
- ✓ Purchase Compute Savings Plans (not EC2 SPs) for Fargate + Lambda workloads — flexibility applies discount across all compute types regardless of region or size
- ✓ Use S3 Gateway Endpoints for S3 and DynamoDB traffic from private subnets — completely free; immediately eliminates NAT Gateway data processing charges for those services
- ✓ Set log retention on all CloudWatch Log Groups — use a CDK Aspect to enforce 30-day default; prevents indefinite log accumulation (most common overlooked cost)
- ✓ Use AWS Budgets with forecasted alerts — get notified when spend is FORECAST to exceed budget (not just when it already has); allows corrective action before month-end
- ✓ Use Spot Instances only for fault-tolerant batch workloads — Glue ETL, CodeBuild, dev ECS; never for production stateful services that cannot tolerate 2-minute interruption notice
- ✓ Enable S3 Intelligent Tiering for data lake Bronze layer — access patterns for raw data are unknown; Intelligent Tiering auto-optimizes without lifecycle rule tuning; monitoring fee: \$0.0025 per 1,000 objects

■ Never purchase Reserved Instances before right-sizing — buying r6g.2xlarge RIs for an over-provisioned service that should be r6g.xlarge locks in 2x cost for 1-3 years

■ Do not rely on Trusted Advisor alone for cost optimization — Trusted Advisor checks EC2 utilization but misses Fargate over-provisioning, log retention, NAT Gateway, and cross-AZ transfer costs

■ Avoid On-Demand mode for DynamoDB tables with predictable, stable traffic — On-Demand costs 1.25x more per request than Provisioned; only use On-Demand for genuinely variable or unpredictable workloads

■ Do not enable VPC Flow Logs on all ENIs without a retention policy — Flow Logs generate 100s of GBs per month for a busy VPC; set 7-day CloudWatch retention + S3 export for longer-term Athena analysis

19-QR

Quick Reference and Interview Q&A

Concept	Key Value / Rule
Savings Plans vs Reserved Instances	Savings Plans (SP): commit to \$/hour spend; flexible across instance families, regions, sizes; apply to EC2 + Fargate
Fargate right-sizing formula	Right-sized CPU = P95 CPU utilization x 1.5 (50% headroom). Right-sized Memory = P95 Memory x 1.5. Then round up to nearest size.
S3 storage class decision	Standard: >1 access/month. Standard-IA: <1 access/month, retrieval acceptable, object >128 KB. Intelligent-Tiering: mixed access patterns.
NAT Gateway vs VPC Endpoints	NAT Gateway: \$0.045/GB processed (both directions). S3 Gateway Endpoint: \$0/GB (free). DynamoDB Gateway Endpoint: \$0.00013/GB processed.
DynamoDB On-Demand vs Provisioned	On-Demand: \$1.25 per million WCU, \$0.25 per million RCU. Provisioned: \$0.00065 per WCU-hour, \$0.00013 per RCU-hour.
Cost Anomaly Detection vs Budgets	Budgets: threshold-based (alert when spend > \$X or > Y% of budget); useful for total monthly spend monitoring.